Projeto: Banco App

Neste projeto você vai implementar um **sistema bancário** simples em Python, com cadastro de titulares, abertura e exclusão de contas, depósitos e saques. O projeto aplica os conceitos de orientação a objetos: encapsulamento, atributos privados, propriedades com validação, associação entre classes, geradores e exceções.

O projeto é dividido em três partes:

1. **Modelo** (`account.py`, `holder.py`, `bank.py`, `exceptions.py`): as regras do banco, sem nenhum `print`, `input`, código de interface ou acesso a arquivos.
2. **Persistência** (`database.py`): salva o banco em um arquivo JSON e o carrega de volta. É a **única** parte do projeto que sabe que existe um arquivo.
3. **Interfaces** (`bank_terminal.py` e `bank_gui.py`): mostram os titulares e as contas, repassam as operações para o modelo e pedem para a persistência salvar depois de cada mudança.

Como o modelo não depende de interface nem de arquivo, as duas interfaces usam exatamente as mesmas classes, e daria para trocar o JSON por um banco de dados sem mexer no modelo.

As classes e métodos apresentados neste documento são apenas sugestões, você pode criar a estrutura que achar melhor. A única exigência é que as regras do banco fiquem separadas da interface e da persistência.

Regras do banco

- Um **titular** tem nome e CPF, os dois obrigatórios. Não pode haver dois titulares com o mesmo CPF no banco.
- Um titular pode ter **várias contas**, e cada conta pertence a **um único** titular.
- Cada conta tem um **número** e um **saldo**. Não pode haver duas contas com o mesmo número no banco, mesmo que sejam de titulares diferentes. O número de uma conta nova é escolhido pelo banco: é o maior número de conta existente mais 1. O saldo de uma conta nova é zero.
- Um **depósito** precisa ter valor positivo.
- Um **saque** precisa ter valor positivo e não pode ser maior que o saldo. Ou seja, o saldo **nunca** fica negativo.
- Um titular pode excluir qualquer uma das suas contas.

Toda regra violada lança uma exceção do banco (veja Exceções), com uma mensagem que explica o problema. As interfaces mostram essa mensagem para o usuário.

Titular e conta: uma associação em um sentido só

O titular conhece as suas contas, mas a conta **não** conhece o seu titular:

```
holder.get_account(1)    # titular → conta: existe
account.holder           # conta → titular: não existe
```

Ao modelar uma associação, é preciso escolher em que sentido ela pode ser **navegada**. Aqui, todas as operações começam por um titular: o usuário escolhe o titular e só depois uma das suas contas. Por isso basta o sentido titular → conta. A conta é uma **parte** do titular (uma composição), e não precisa saber a quem pertence.

Uma associação nos dois sentidos (**bidirecional**) também funcionaria, mas teria um custo:

- **Os dois lados precisariam concordar.** Não poderia existir uma conta que diz pertencer à Maria guardada na lista de contas do João, e o código teria que impedir isso.
- **Os módulos dependeriam um do outro.** `holder.py` importaria `account.py`, e `account.py` importaria `holder.py`, um **import circular**.

Com a associação em um sentido só, nenhum dos dois problemas existe. A regra "cada conta pertence a um único titular" continua valendo, garantida pelo `Bank`: é ele quem cria as contas, e cada conta nova vai para um titular só.

Persistência separada do modelo

O modelo não sabe que os dados vão para um arquivo. Quem salva e carrega é a classe `Database`, e quem decide **quando** salvar é a interface: depois de cada operação que muda o banco, ela chama `database.save(bank)`.

```
interface ──── operação ────→ modelo (Bank, Holder, Account)
    │
    └──── save(bank) ────→ Database ──→ bank.json
```

Para carregar, o `Database` lê o JSON e **reconstrói** o banco usando os métodos públicos do `Bank` (`create_holder` e `add_account`). Assim, as regras também valem para os dados lidos do arquivo: um JSON com dois titulares de mesmo CPF, ou com duas contas de mesmo número, lança `DuplicateHolderError` ou `DuplicateAccountError` ao ser carregado.

O arquivo tem este formato:

```
[
  {
    "name": "Maria Silva",
    "cpf": "111.111.111-11",
    "accounts": [
      {"number": 1, "balance": 350.0}
    ]
  }
]
```

Classes

Exceções (`exceptions.py`)

Classe	Responsabilidade
`BankError(Exception)`	Exceção base de todos os erros do banco.
`InvalidDataError(BankError)`	Lançada quando um dado obrigatório está vazio (nome, CPF) ou fora do permitido (número de conta não positivo, saldo inicial negativo).
`InvalidAmountError(BankError)`	Lançada quando o valor de um depósito ou saque não é positivo.
`InsufficientFundsError(BankError)`	Lançada quando o valor do saque é maior que o saldo.
`DuplicateHolderError(BankError)`	Lançada quando já existe um titular com o mesmo CPF.
`DuplicateAccountError(BankError)`	Lançada quando já existe uma conta com o mesmo número.
`HolderNotFoundError(BankError)`	Lançada quando o titular não pertence ao banco.
`AccountNotFoundError(BankError)`	Lançada quando o titular não tem uma conta com o número informado.

Como todas herdam de `BankError`, as interfaces tratam qualquer erro do banco com um único `except BankError`. Uma exceção específica fica disponível para quem precisar reagir a um erro em particular, por exemplo, oferecer um valor menor quando o saque falha por `InsufficientFundsError`.

`Account` (`account.py`)

Representa **uma conta bancária**.

- `__init__(number: int, balance: float = 0.0)`: lança `InvalidDataError` se o número não for positivo ou se o saldo for negativo.
- Propriedades somente leitura `number` e `balance`. O saldo só muda por meio de `deposit` e `withdraw`.
- `deposit(amount)`: soma `amount` ao saldo. Lança `InvalidAmountError` se o valor não for positivo.
- `withdraw(amount)`: subtrai `amount` do saldo. Lança `InvalidAmountError` se o valor não for positivo e `InsufficientFundsError` se for maior que o saldo.
- `__str__()`: o número da conta com 4 dígitos, como `0007`.

`Holder` (`holder.py`)

Representa **um titular**, com as suas contas.

- `__init__(name: str, cpf: str)`: começa sem nenhuma conta.
- `name` e `cpf`: propriedades com **setter**. Os setters lançam `InvalidDataError` se o valor for vazio. O próprio `__init__` usa os setters (`self.name = name`), para que a validação valha também na criação.
- `add_account(account)`: guarda a conta. Lança `DuplicateAccountError` se o titular já tiver uma conta com aquele número.

- `delete_account(account_number)`: remove a conta. Lança `AccountNotFoundError` se o titular não tiver uma conta com aquele número.
- `get_account(account_number) -> Account`: a conta com aquele número. Lança `AccountNotFoundError` se o titular não tiver essa conta.
- `accounts()`: **gerador** que produz as contas do titular, uma por vez.

As contas ficam em um dicionário `__accounts`, cuja chave é o número da conta. Assim, buscar, excluir e conferir se um número já existe não exige percorrer todas as contas.

> Dica: um método que usa `yield` é um gerador. `yield from self.__accounts.values()` produz cada conta, uma por vez, sem expor o dicionário: quem recebe o gerador consegue percorrer as contas, mas não consegue acrescentar nem remover nenhuma.

`Bank` (`bank.py`)

É o **cadastro do banco**: guarda os titulares e escolhe os números das contas.

- `__init__()`: começa sem nenhum titular.
- `create_holder(name, cpf) -> Holder`: cria e cadastra um titular. Lança `DuplicateHolderError` se já existir um titular com aquele CPF.
- `create_account(holder) -> Account`: cria uma conta com saldo zero, com o próximo número disponível, e a acrescenta ao titular usando `add_account`.
- `add_account(holder, account)`: acrescenta ao titular uma conta que **já tem número**. Lança `HolderNotFoundError` se o titular não for deste banco e `DuplicateAccountError` se já existir, em qualquer titular, uma conta com aquele número. É usado por `create_account` e pelo `Database`, ao carregar as contas do arquivo.
- `holders()`: **gerador** que produz todos os titulares.
- `filter_by_name(name)`: **gerador** que produz os titulares cujo nome **contém** `name`, sem diferenciar maiúsculas de minúsculas. Um texto vazio produz todos os titulares.
- `__next_account_number() -> int`: **método privado** que calcula o próximo número de conta: o maior número existente mais 1, ou 1 se ainda não houver contas.

As contas entram pelo **banco**, não direto pelo titular, porque só o banco conhece as contas de todos os titulares e consegue garantir que os números não se repitam. O `Holder.add_account` só confere as contas do próprio titular.

`Database` (`database.py`)

Salva e carrega o banco em um arquivo JSON.

- `__init__(path: Path = DEFAULT_PATH)`: o caminho do arquivo. Por padrão, é o `bank.json` na mesma pasta do código.
- `path -> Path`: o caminho do arquivo.
- `load() -> Bank`: lê o arquivo e reconstrói o banco. Se o arquivo ainda não existir, retorna um banco vazio.
- `save(bank)`: grava todos os titulares e contas do banco no arquivo, substituindo o conteúdo anterior.

> Dica: use `json.dump(data, f, ensure_ascii=False, indent=2)` para os acentos ficarem legíveis no arquivo e cada informação ficar em uma linha.

Interfaces

As interfaces **não implementam regra nenhuma** e **não acessam o arquivo diretamente**. Elas só mostram os dados, chamam as operações do modelo, tratam o `BankError` mostrando a mensagem para o usuário e chamam `database.save` depois de cada mudança.

As interfaces também convertem o texto digitado em número, com `int()` ou `float()`, e essas funções lançam `ValueError` se o texto não for um número. Esse erro é tratado à parte, com uma mensagem própria ("digite um valor numérico"), porque é um erro de digitação e não uma regra do banco.

`bank_terminal.py`: classe `BankTerminal`, com um sistema de menus numerados:

- **Menu principal:** cadastrar um titular novo ou buscar um titular por parte do nome (Enter lista todos).
- **Menu do titular:** listar as contas, criar uma conta ou excluir uma conta pelo número.
- **Menu da conta:** depositar ou sacar. O valor aceita vírgula ou ponto como separador decimal.

Os menus usam o método genérico `choose[T](items: list[T]) -> T | None`, que mostra uma lista numerada e devolve o item escolhido, ou `None` se o usuário apertar Enter ou digitar um número inválido. O mesmo método serve para escolher um titular ou uma conta.

`bank_gui.py`: classe `BankApp`, que herda de `tk.Tk`, com dois painéis lado a lado:

- **Titulares:** um campo de busca, que filtra a lista enquanto o usuário digita, a lista de titulares (em um `ttk.Treeview`) e os campos para cadastrar um titular novo.
- **Contas:** as contas do titular selecionado, os botões **Nova conta** e **Excluir conta** (com confirmação) e um campo de valor com os botões **Depositar** e **Sacar**.

Uma barra de status embaixo mostra o resultado da última operação, e os erros aparecem em uma caixa de mensagem (`messagebox`).

Diagrama de classes



A seta entre `Holder` e `Account` aponta só para `Account`: o titular conhece as contas, mas a conta não conhece o titular.

Ordem sugerida de implementação

1. `exceptions.py` e `Account`. Teste depósitos e saques válidos e confira se os inválidos (valor negativo, saque maior que o saldo) lançam `InvalidAmountError` ou `InsufficientFundsError` sem alterar o saldo.
2. `Holder`, com as validações de nome e CPF. Crie um titular, acrescente contas e confira se `add_account` recusa um número de conta repetido.

3. `Bank`. Crie alguns titulares e contas e confira se os números das contas não se repetem, se o CPF repetido é recusado e se `add_account` recusa, para um titular, o número de uma conta de outro titular.
4. `Database`. Salve um banco, abra o `bank.json` para conferir o conteúdo e carregue de novo. O banco carregado precisa ter os mesmos titulares, contas e saldos.
5. `bank_terminal.py`, para usar o sistema e testar as regras.
6. `bank_gui.py`.